

CHAPTER - 3

Context Free Language and Pushdown Automata

3.1 Context Free Grammar (CFG)

The context free grammar is a **language generator** which operates on some set of rules called the production rule. The context-free languages are applied in *parser design*. They are also useful for describing *block structure of programming languages*. In the production rule of context sensitive language, the replacement of any non-terminal symbol have the influence of their respective terminal but in case of the context free language, the non-terminals or variables are replaced independently as we have to *define the production rule for each non-terminals individually*.

A. In Production Rule of Context Free Grammar (CFG)

Non-terminal $\rightarrow$ Terminals only or Non-terminals only or Combination of terminals and non-terminals

For example: $s \rightarrow a/b/SS/MM$, $M \rightarrow p/D/q$, $D \rightarrow x/y/z$ etc. This implies that each non-terminal is separately or individually defined so they are context free on their replacement.

B. In Production Rule of Context Sensitive Grammar (CSG)

Non-terminal only or Combination of non-terminals and terminals $\rightarrow$ Terminals only or Non-terminals only or Combination of terminals and non-terminals

For example: $aaSbc \rightarrow aab/Sbb/aaS$, $S \rightarrow aa/bb$ etc. In the first production rule, it implies that on replacing the $aaSbc$, we have to consider the present context of S present ahead and back side also.

Mathematically, a grammar $G = (V, \Sigma, R, S)$ is a context-free grammar (CFG) if

V = Finite set of non-terminals or variables that are represented by capital letters

Σ = Finite set of terminals that are represented by the small letter or sign or number etc

S = Starting non-terminal symbol and $S \in V$

R = Set of rules called the production rule of the form $\alpha \rightarrow \beta$, where $\alpha \in V$ and $\beta \in (V \cup \Sigma)^*$ (i.e. the LHS of production rule in CFG have only the non-terminals and the RHS may have empty string, terminals, non-terminals or the combination of terminals and non-terminals).

The production rules or productions or rewriting rules is the kernel of any grammar and language specification. The productions are used to derive one string over $V \cup \Sigma$ from another string. In the application of the production rule, the reverse substitution is not permitted i.e. if $S \rightarrow AA$ then $AA \rightarrow S$ is not possible.

Note: Is there any relationship between the regular expression and the context free grammar? Define by suitable example.

Solution

We know from the Chomsky hierarchy of grammar, all the regular expressions can be described on the basis of the Context Free Grammar (CFG), but the reverse cannot be true. This can be verified with the following example. Let any regular expression, $R = a(a^*b^*)b$, then the operating rules for R can be generated as:

$S \rightarrow aMb$ (i.e. S = start state = String with M starting with a and ending with b)
 $M \rightarrow A/B$ [i.e. $M = (a^*+b^*)$ = any number of string of a or b]
 $A \rightarrow aA/\wedge$ (i.e. String with any number of a)
 $B \rightarrow bB/\wedge$ (i.e. String with any number of b)

Now,

Let $G = (V, \Sigma, R, S)$ be a context-free grammar (CFG) that can describe the string generated by regular expression, R . Where

V = Finite set of non-terminals or variables = $\{S, M, A, B\}$

Σ = Finite set of terminals = $\{a, b, \wedge\}$

S = Starting non-terminal symbol and $S \in V$

R = The production rule, which can be describe as: $S \rightarrow aMb$, $M \rightarrow A/B$, $A \rightarrow aA/\wedge$, & $B \rightarrow bB/\wedge$.

Numerical – 1: Write a CFG to generate only the palindrome with the input symbol, $\Sigma = \{0, 1\}$.

Solution

Let $G = (V, \Sigma, R, S)$ be a context-free grammar (CFG) that can describe the string of palindrome only.

Where V = Finite set of non-terminals or variables = $\{S\}$

Σ = Finite set of terminals = $\{0, 1, \wedge\}$

S = Starting non-terminal symbol and $S \in V$

R = Production rule, which can be describe as: $S \rightarrow 0S0/1S1/\wedge$.

Note: The production rule will be $S \rightarrow 0S0/1S1/\wedge$ or $S \rightarrow SS/1/0/\wedge$ for the palindrome and non-palindrome string generation but not only the palindrome as above.

Derivations

The process of deriving the required string over $(V \cup \Sigma)^*$ using the given production rule on the existing string is called the derivation. The string generated by the most recent application of production is called the *working string*. The derivation of a string completed when the working string cannot be modified. The different derivations results are quite different in different sentential form such as *context sensitive grammar*, but for a context free grammar, it really doesn't make much difference in what order you expand the variable.

Suppose that $\alpha_1, \alpha_2 \dots \alpha_m$ are strings over $(V \cup \Sigma)^*$ and $\alpha_1 \xrightarrow{G} \alpha_2 \xrightarrow{G} \alpha_3 \xrightarrow{G} \alpha_4 \dots \xrightarrow{G} \alpha_{(m-1)} \xrightarrow{G} \alpha_m$. Then, we say that α_1 derives α_m in grammar G or this can be represented as $\alpha_1 \xrightarrow{*G} \alpha_m$.

Hence, we call the sequence of the derivation in G of α_m from α_1 in the following manner as: $\alpha_1 \xrightarrow{G} \alpha_2 \xrightarrow{G} \alpha_3 \xrightarrow{G} \alpha_4 \dots \xrightarrow{G} \alpha_m$

The derivations are described as following manner.

- If we describe a string by applying the production rule at once (i.e. one time application) then the string is called **directly derivable string** and is denoted by $\alpha \xrightarrow{G} \beta$
- If we derive the string by more than one sequence of operations using given production rules then the string is called **derivable string** and is denoted by $\alpha \xrightarrow{*G} \beta$ or $\alpha \xrightarrow{*} \beta$

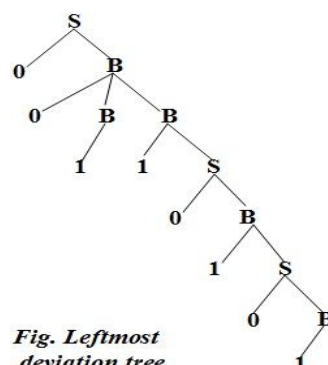
Type of derivation

A. Leftmost derivation

A derivation $A \xrightarrow{*} w$ is called a leftmost derivation if we apply a production only to the leftmost variable at every step. For example, if the given production rules of any CFG are as: $S \rightarrow 0B/1A$, $A \rightarrow 0/0S/1AA$, & $B \rightarrow 1/1S/0BB$ then any given string $w = 00110101$ can be derived using the leftmost derivation in following manner:

$S \rightarrow 0B$
 $\rightarrow 00BB$ (As, $B \rightarrow 0BB$)
 $\rightarrow 001B$ (As, $B \rightarrow 1$)
 $\rightarrow 0011S$ (As, $B \rightarrow 1S$)
 $\rightarrow 00110B$ (As, $B \rightarrow 0B$)
 $\rightarrow 001101S$ (As, $B \rightarrow 1S$)
 $\rightarrow 0011010B$ (As, $B \rightarrow 0B$)
 $\rightarrow 00110101$ (As, $B \rightarrow 1$)

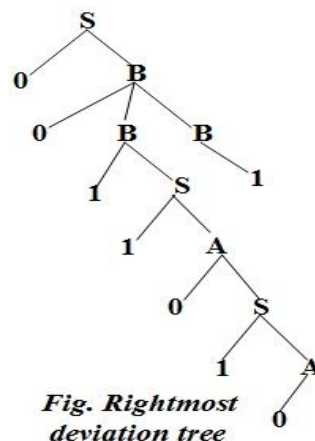
Here, the tree is a derivation tree with yield 00110101.



B. Rightmost derivation

A derivation $A \xrightarrow{*} w$ is called a rightmost derivation if we apply a production only to the rightmost variable at every step. For example, if the given production rules of any CFG are as: $S \rightarrow 0B/1A$, $A \rightarrow 0/0S/1AA$, & $B \rightarrow 1/1S/0BB$ then any given string $w = 00110101$ can be derived using the rightmost derivation in following manner:

$S \rightarrow 0B$
 $\rightarrow 00BB$ (As, $B \rightarrow 0BB$)
 $\rightarrow 00B1$ (As, $B \rightarrow 1$)
 $\rightarrow 001S1$ (As, $B \rightarrow 1S$)
 $\rightarrow 0011A1$ (As, $S \rightarrow 1A$)
 $\rightarrow 00110S1$ (As, $A \rightarrow 0S$)
 $\rightarrow 001101A1$ (As, $S \rightarrow 1A$)
 $\rightarrow 00110101$ (As, $A \rightarrow 0$)



3.2 Representation of CFG

The Context Free Grammar (CFG) can be represented in two ways:

- Derivation tree or Parse tree or Production tree
- Backus Naur Form (BNF)

A. Derivation Tree

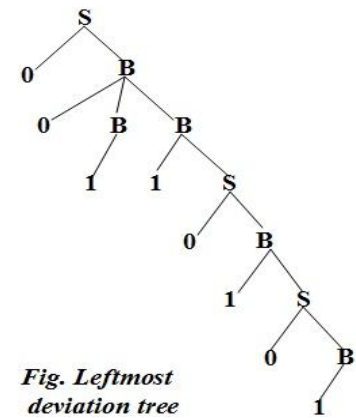
It is easy to visualize derivation in context-free languages as we can represent derivations using tree structure. Such tree representing derivations are called derivation trees or *parse trees*. A parse tree is an ordered tree in which nodes are labeled with the left side of production (i.e. non-terminals only) and the children of the nodes (i.e. leaves) represent its corresponding right-sides (i.e. the terminals or non-terminals or both).

A derivation tree for a CFG, $G = (V, \Sigma, R, S)$ is a tree satisfying the following conditions:

1. Every vertex has label, which is a variable or terminal or empty string ($\wedge$).
2. The root has label 'S' (i.e. start symbol).
3. The label of an internal vertex is a variable.
4. Each vertex of variable is extended towards the leaf-node or terminals using the production rule (R).

For example, if the given production rules of any CFG are as:
 $S \rightarrow 0B/1A$, $A \rightarrow 0/0S/1AA$, & $B \rightarrow 1/1S/0BB$ then any given string $w = 00110101$ can be derived using the leftmost derivation in following manner: -

$S \rightarrow 0B$
 $\rightarrow 00BB$ (As, $B \rightarrow 0BB$)
 $\rightarrow 001B$ (As, $B \rightarrow 1$)
 $\rightarrow 0011S$ (As, $B \rightarrow 1S$)
 $\rightarrow 00110B$ (As, $B \rightarrow 0B$)
 $\rightarrow 001101S$ (As, $B \rightarrow 1S$)
 $\rightarrow 0011010B$ (As, $B \rightarrow 0B$)
 $\rightarrow 00110101$ (As, $B \rightarrow 1$)



Here, the tree is a derivation tree with yield 00110101. The yield of a derivation tree is the concatenation of the labels of the leaves without repetition in the left-to-right ordering.

Exercise

- Consider a CFG, $S \rightarrow XX$, & $X \rightarrow XXX/bX/Xb/a$, then find the parse tree for any given string $w = bbaaab$.
- Consider the grammar G , with production $S \rightarrow aXY$, $X \rightarrow bYb$, & $Y \rightarrow X/c$, then find the parse tree for any string $w = abbbb$.

Note: The derivation tree does not specify the order in which we apply the production for getting the required string. So, same derivation tree can include several derivations. But, in general we use leftmost derivation than that of the rightmost derivation.

3.3 Ambiguity in Context-Free Grammar

Grammars are used to put structures on programs or documents. The assumption was that a grammar uniquely determines a structure for each string in the language. However, not every grammar does provide unique structure. Thus, when a grammar fails to provide unique structure, it is called the ambiguous grammar i.e. in this case grammar puts more than one structure for same string in the language.

A Context Free Grammar, G is ambiguous if there exists some terminal string $w \in L(G)$ is ambiguous. The terminal string w is ambiguous if there exists two or more leftmost derivations for single w . In other word, the single terminal string w is ambiguous if it may be the yield of two derivation trees.

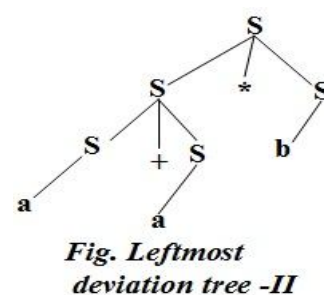
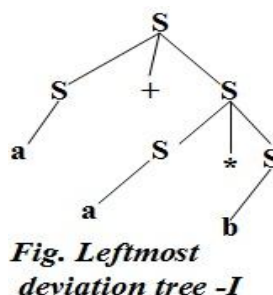
For example: Consider $G = (\{S\}, \{a, b, +, *\}, R, S)$, where R consists of $S \rightarrow S+S/S*S/a/b$. Now, we have two derivation trees for the terminal string $w = a + a * b$ as given below: -

Leftmost derivation – I

$S \rightarrow S+S$
 $\rightarrow a + S*S$
 $\rightarrow a + a * S$
 $\rightarrow a + a * b$

Leftmost derivation – II

$S \rightarrow S*S$
 $\rightarrow S + S*S$
 $\rightarrow a + S * S$



$\rightarrow a + a * S$
 $\rightarrow a + a * b$

Since, here exists two leftmost derivation trees for a same terminal string $w = a + a * b$ (i.e. w is ambiguous). Hence, we can conclude that the given grammar G is ambiguous.

Exercise

- Consider any grammar G , with the production rule $S \rightarrow SbS/a$. Show that the given grammar is ambiguous. Assume the terminal string $w = abababa$.

3.4 Normal Form

Since, in CFG, the production rule is of the form $\alpha \rightarrow \beta$, where $\alpha \in V$ and $\beta \in (V \cup \Sigma)^*$ (i.e. the LHS of production rule in CFG have only the non-terminals and the RHS may have empty string, terminals, non-terminals or the combination of terminals and non-terminals). Thus, when we apply some restriction on the R.H.S. of the Context Free Grammar G for defining new production then G is said to be in “normal form”. Among several normal forms, we deal with the following two normal forms:

- Chomsky Normal Form (CNF)
- Greibach Normal Form (GNF)

A. Chomsky Normal Form (CNF)

In the Chomsky normal form, we have restrictions on the length of R.H.S. and the nature of symbols in the R.H.S. of production. Here, the restriction is that every node has either two internal vertices (i.e. two non-terminals only) or a single leaf (i.e. exactly one terminal). When a grammar is in CNF, some of the proofs and constraints are simpler.

A context-free grammar G is in Chomsky normal form if every production is of the form $A \rightarrow a$ and $A \rightarrow BC$. Here, A , B and C are non-terminal symbols, ‘ a ’ is a terminal symbol, S is the start symbol, and ϵ is the empty string. Also, neither B nor C may be the start symbol but A may be. For **example**: consider G whose productions are $S \rightarrow AB$, $A \rightarrow a$, & $B \rightarrow c$. Then G is in CNF.

Note: Any Context Free Grammar (CFG) can be reduced into the Chomsky normal form (CNF) but the converse is not true as CNF is a restricted form of CFG.

Reduction to Chomsky Normal Form

Any context-free language is generated by a context-free grammar in Chomsky normal form. For every context-free grammar (CFG), there is an equivalent grammar G_2 in Chomsky normal form (CNF). Here, we put the non-terminals in place of terminals in the same principle of the *bottom-up parsing*.

Proof idea:

- Show that any CFG G can be converted into a CFG G' in Chomsky normal form;
- Conversion procedure has several stages where the rules that violate Chomsky normal form conditions are replaced with equivalent rules that satisfy these conditions;
- Order of transformations: (1) add a new start variable, (2) eliminate all null production, (3) eliminate unit-productions, (4) Elimination of terminals on R.H.S.
- Check that the obtained CFG G' define the same language as the initial CFG G by restricting the number of variables on R.H.S.

For example: If the given production rules of context-free grammar, G is given as: $S \rightarrow aAD$, $A \rightarrow aB/bAB$, $B \rightarrow b$, & $D \rightarrow d$, then construct the equivalent CNF.

Solution

Since in CNF, the restriction is that every nodes on R.H.S. has either two internal vertices (i.e. two non-terminals only) or a single leaf (i.e. exactly one terminal). Also, there are no null productions or unit production. Thus, the production rule can be constructed as: -

- $B \rightarrow b$ & $D \rightarrow d$ are in R' .
- $S \rightarrow aAD$ gives $S \rightarrow C_aAD$, where $C_a \rightarrow a$ and also $S \rightarrow C_aAD$, gives $S \rightarrow C_aC_1$, where $C_1 \rightarrow AD$.
- $A \rightarrow aB$ gives $A \rightarrow C_aB$, where $C_a \rightarrow a$
- $A \rightarrow bAB$ gives $A \rightarrow BAB$, where $B \rightarrow b$ and also $A \rightarrow BAB$ gives $A \rightarrow BC_b$, where $C_b \rightarrow AB$.

Hence, Let $G' = (V', \Sigma, R', S')$ be newly constructed CNF equivalent to the given CFG, where

$V' = \text{Set of non-terminals only} = \{S, A, B, D, C_a, C_b, C_1\}$

$\Sigma = \text{Set of terminals only} = \{a, b, d\}$

$S' = \text{Start state} = S$

$R' = \text{Set of production rule that can be defined as: } S \rightarrow C_aC_1, A \rightarrow C_aB, A \rightarrow BC_b, C_a \rightarrow a, C_b \rightarrow AB, C_1 \rightarrow AD, B \rightarrow b \text{ \& } D \rightarrow d.$

For example: If the given production rules of context-free grammar, G is given as: $S \rightarrow aBASA/aBA$, $A \rightarrow aAA/a$, & $B \rightarrow bBB/b$ then construct the equivalent CNF.

Solution

We can construct the CNF for the given CFG by defining the production rule as:

- $B \rightarrow b$ & $A \rightarrow a$.
- $S \rightarrow aBASA$ gives $S \rightarrow C_aBASA$, where $C_a \rightarrow a$
 $S \rightarrow C_aBASA$, gives $S \rightarrow C_aC_1SA$, where $C_1 \rightarrow BA$.
 $S \rightarrow C_aC_1SA$, gives $S \rightarrow C_aC_1C_2$, where $C_2 \rightarrow SA$.
 $S \rightarrow C_aC_1C_2$, gives $S \rightarrow C_aC_3$, where $C_3 \rightarrow C_1C_2$.
 - $S \rightarrow aBA$ gives $S \rightarrow C_aBA$, where $C_a \rightarrow a$
 $S \rightarrow C_aBA$ gives $S \rightarrow C_aC_1$, where $C_1 \rightarrow BA$.
 - $A \rightarrow aAA$ gives $A \rightarrow C_aAA$, where $C_a \rightarrow a$.
 $A \rightarrow C_aAA$ gives $A \rightarrow C_aC_4$, where $C_4 \rightarrow AA$.
 - $B \rightarrow bBB$ gives $B \rightarrow C_bBB$, where $C_b \rightarrow b$.
 $B \rightarrow C_bBB$ gives $B \rightarrow C_bC_5$, where $C_5 \rightarrow BB$.

Hence, Let $G' = (V', \Sigma, R', S')$ be newly constructed CNF equivalent to the given CFG, where

$V' = \text{Set of non-terminals only} = \{S, A, B, C_a, C_b, C_1, C_2, C_3, C_4, C_5\}$

$\Sigma = \text{Set of terminals only} = \{a, b\}$

$S' = \text{Start state} = S$

$R' = \text{Set of production rule that can be defined as: } S \rightarrow C_aC_3/C_aC_1, C_a \rightarrow a, C_1 \rightarrow BA, C_2 \rightarrow SA, C_3 \rightarrow C_1C_2, S \rightarrow C_aC_1, A \rightarrow C_aC_4, C_4 \rightarrow AA, B \rightarrow C_bC_5, C_b \rightarrow b \text{ \& } C_5 \rightarrow BB.$

B. Greibach Normal Form (GNF)

GNF is another normal form quite useful in some proofs and constructions. A context free grammar generating the set accepted by a pushdown is in GNF. A grammar in GNF is a natural generalization of a regular grammar as the production of the regular grammar are of the form $A \rightarrow a\alpha$, where $\alpha \in V^*$ and $a \in \Sigma$.

A context-free grammar is in Greibach normal form if every production is in the form $A \rightarrow a\alpha$, where $\alpha \in V^*$ and 'a' is only one terminal, $a \in \Sigma$. Hence, any CFG will be in GNF if it is in the form "non-terminal $\rightarrow$ exactly one terminal followed by any number of non-terminals including null (ϵ). For example, the grammar G with productions $S \rightarrow aAB$, $A \rightarrow bC$, $B \rightarrow b$, $C \rightarrow \epsilon$ is in GNF.

Conversion of CFG into GNF

Example: If the given production rules of context-free grammar, G is given as: $S \rightarrow abaSa/aba$, then construct the equivalent GNF.

Solution

We can construct the GNF for the given CFG by defining the production rule.

Here, we have

$$S \rightarrow abaSa/aba$$

Now, let us introduce new variables A and B and productions $A \rightarrow a$ & $B \rightarrow b$ and substitute into the given grammar as

$$S \rightarrow aBASA/aBA$$

$$A \rightarrow a \text{ \& }$$

$$B \rightarrow b$$

Hence, Let $G' = (V', \Sigma, R', S')$ be newly constructed GNF equivalent to the given CFG, where

$$V' = \text{Set of non-terminals only} = \{S, A, B\}$$

$$\Sigma = \text{Set of terminals only} = \{a, b\}$$

$$S' = \text{Start state} = S$$

$$R' = \text{Set of production rule that can be defined as: } S \rightarrow aBASA/aBA, A \rightarrow a \text{ \& } B \rightarrow b.$$

Example: If the given production rules of context-free grammar, G is given as: $S \rightarrow AB$, $A \rightarrow aA/bB/b$ & $B \rightarrow b$ then construct the equivalent GNF.

Solution

We can construct the GNF for the given CFG by defining the production rule as: -

- $S \rightarrow AB$ gives $S \rightarrow aAB$, (Since $A \rightarrow aA$)
- $S \rightarrow AB$ gives $S \rightarrow bBB$, (Since $A \rightarrow bB$)
- $S \rightarrow AB$ gives $S \rightarrow bB$, (Since $A \rightarrow b$)
- $A \rightarrow aA/bB/b$
- $B \rightarrow b$

Now, Let $G' = (V', \Sigma, R', S')$ be newly constructed GNF equivalent to the given CFG.

Where

$$V' = \text{Set of non-terminals only} = \{S, A, B\}$$

$$\Sigma = \text{Set of terminals only} = \{a, b\}$$

$$S' = \text{Start state} = S$$

$$R' = \text{Set of production rule that can be defined as: } S \rightarrow aAB/bBB/bB, A \rightarrow aA/bB/b \text{ \& } B \rightarrow b.$$

3.5 Simplification of CFG

The unit production, useless productions & the useless production in CFG makes it ambiguous & thus there is not any unique structure for any language generated by using the grammar and also the cost of production will increase. Thus, for simplification of CFG, we attempt to do following procedures:

- Removal of unit productions
- Removal of null productions
- Removal of useless productions

A. Elimination of Unit Productions

A Context Free Grammar G may have production of the form $A \rightarrow B$, where A & B are variable in G , (i.e. one non-terminal $\rightarrow$ one non-terminal) is called unit production. On removing the unit productions, we analyze the non-terminals by substitution of the terminals. The unit production increases the cost of the derivation in grammar.

Example: If the given production rules of context-free grammar, G is given as: $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow C/b$, $C \rightarrow D$, $D \rightarrow E$ & $E \rightarrow a$, then remove the unit productions.

Solution

The given grammar contain the following unit productions

$B \rightarrow C$,
 $C \rightarrow D$, &
 $D \rightarrow E$

Also, the terminals given by the non-terminals are

$A \rightarrow a$,
 $B \rightarrow b$, gives $C \rightarrow b$ and hence it generates $B \rightarrow b/b$ or $B \rightarrow b$ which is given
 $E \rightarrow a$, gives $D \rightarrow a$ & hence $C \rightarrow a$ and finally we get $B \rightarrow a$

Now, the useless production for the given CFG can be distinguished by analyzing the not reachable non-terminals from the start symbol. Here, the non-terminals are A & B are only reachable from start symbol and others are useless productions i.e. $C \rightarrow b$, $E \rightarrow a$ & $D \rightarrow a$ are useless or never be used. Hence, we can eliminate them.

Now, Let $G' = (V', \Sigma, R', S')$ be newly constructed CFG, which is completely reachable grammar.

Where

V' = Set of non-terminals only = $\{S, A, B\}$

Σ = Set of terminals only = $\{a, b\}$

S' = Start state = S

R' = Set of production rule that can be defined as: $S \rightarrow AB$, $A \rightarrow a$ & $B \rightarrow a/b$.

Example: If the given production rules of context-free grammar, G is given as: $S \rightarrow A/bb$, $A \rightarrow B/b$, & $B \rightarrow S/a$, then remove the unit productions.

Solution

The given grammar contain the following unit productions

$S \rightarrow A$,
 $A \rightarrow B$, &
 $B \rightarrow S$

Also, the terminals given by the non-terminals are

$B \rightarrow a$, gives $S \rightarrow a$ & hence $A \rightarrow a$
 $A \rightarrow b$, gives $S \rightarrow b$ & hence $B \rightarrow b$

$S \rightarrow bb$, gives $B \rightarrow bb$ & hence $A \rightarrow bb$

Now, Let $G' = (V', \Sigma, R', S')$ be newly constructed CFG, which holds no unit productions.

Where

$V' = \text{Set of non-terminals only} = \{S, A, B\}$

$\Sigma = \text{Set of terminals only} = \{a, b\}$

$S' = \text{Start state} = S$

$R' = \text{Set of production rule that can be defined as: } S \rightarrow a/b/bb, A \rightarrow a/b/bb \text{ \& } B \rightarrow a/b/bb.$

Here, the productions of the non-terminals A and B are useless as they are not included on the start symbol or we can say that we can generate the entire possible string only by using the start symbol so we can eliminate the production A & B. Hence the final production rule is only $S \rightarrow a/b/bb$.

B. Elimination of Null Productions

A CFG may have production of the form $A \rightarrow \wedge$, where A is any variable, is called a null production. The production $A \rightarrow \wedge$ is just used to erase A.

To eliminate the null production, we use the following procedure. If $A \rightarrow \wedge$ is a production to be eliminate then we look for all productions, whose right side contains 'A' and replace each occurrence of 'A' in each of these productions to obtain the non-null productions only. Now, these resultant non-null productions must be added to the grammar.

Example: If the given production rules of context-free grammar, G is given as: $S \rightarrow aA$, & $A \rightarrow b/\wedge$, then remove the null productions.

Solution

The given grammar has null productions $A \rightarrow \wedge$. So, put null ($\wedge$) in place of 'A' at the right side of productions and add the resultant productions to the grammar.

Thus,

$S \rightarrow aA$, gives $S \rightarrow a/\wedge$ & hence $S \rightarrow a$

Hence, Let $G' = (V', \Sigma, R', S')$ be newly constructed null-free CFG.

Where

$V' = \text{Set of non-terminals only} = \{S, A\}$

$\Sigma = \text{Set of terminals only} = \{a, b\}$

$S' = \text{Start state} = S$

$R' = \text{Set of production rule that can be defined as: } S \rightarrow a, \text{ \& } A \rightarrow b.$

Here, the production of the non-terminals A is useless as they are not included on the start symbol. Hence the final production rule is only $S \rightarrow a$.

Example: If the given production rules of context-free grammar, G is given as: $S \rightarrow ABAC$, $A \rightarrow aA/\wedge$, $B \rightarrow bB/\wedge$, & $C \rightarrow c$ then remove the null productions.

Solution

The given grammar contains the following null productions:

$A \rightarrow \wedge$

$B \rightarrow \wedge$

So, put null ($\wedge$) in place of 'A' at the right side of productions, we get

$S \rightarrow BAC$, $S \rightarrow ABC$, $S \rightarrow BC$, and $A \rightarrow a$

Thus, the simplified grammar become: $S \rightarrow ABAC/BAC/ABC/BC$, and $A \rightarrow a$

Again, put null ($\wedge$) in place of 'B' at the right side of productions, we get

$S \rightarrow AAC$, $S \rightarrow AC$, $S \rightarrow C$, and $B \rightarrow b$

Thus, the final simplified grammar become: $S \rightarrow ABAC/BAC/ABC/BC/AAC/AC/C$, $A \rightarrow a$ and $B \rightarrow b$

Hence, Let $G' = (V', \Sigma, R', S')$ be newly constructed null-free CFG.

Where

$V' = \text{Set of non-terminals only} = \{S, A, C\}$

$\Sigma = \text{Set of terminals only} = \{a, b\}$

$S' = \text{Start state} = S$

$R' = \text{Set of production rule as: } S \rightarrow ABAC/BAC/ABC/BC/AAC/AC/C, A \rightarrow a \text{ and } B \rightarrow b$

Exercise: Consider the following grammar and remove the null productions.

- $S \rightarrow aSa / bSb / \wedge$
- $S \rightarrow a / Xb / aYa / \wedge, X \rightarrow Y / \wedge, Y \rightarrow b / \wedge$.

C. Elimination of Useless Productions

For the identification of the useful production, the following two points should be noted: -

- a. Can a production generate a string or terminal symbol? This means that not generating is useless productions.
- b. Can a non-terminal symbols (or Variables) are reachable from the start symbol? This means that non reachable are useless productions.

Example: Eliminate the useless productions from context-free grammar, G where $V = \{S, A, B, C\}$ and $\Sigma = \{a, b\}$ with the productions $S \rightarrow aS/A/C$, $A \rightarrow a$, $B \rightarrow aa$, & $C \rightarrow aCb$.

Solution

→ First identify the set of variables that can lead to a terminal string.

i.e. $A \rightarrow a$,

$B \rightarrow aa$, and

$S \rightarrow aS/A$

Since, C cannot generate any string so we remove C . Now, we get a new context-free grammar, G_1 having $V_1 = \{S, A, B\}$ and $\Sigma_1 = \{a\}$ with the production rule R_1 defined as: $S \rightarrow aS/A$, $A \rightarrow a$, $B \rightarrow aa$.

→ Next step is the elimination of the variables that cannot be reached from the start variable or symbol. For this, we draw a dependency graph (or transition diagram) where its vertices are labelled with non-terminals and an edge between C and D is connected if and only if there is a production of the form $C \rightarrow xDy$.

Here, the non-terminal B is useless. Hence the specified grammar is $G' = (V', \Sigma', R', S')$

Where

$V' = \text{Set of non-terminals only} = \{S, A\}$

$\Sigma' = \text{Set of terminals only} = \{a\}$

$S' = \text{Start state} = S$

$R' = \text{Set of production rule as: } S \rightarrow aS/A, A \rightarrow a$.

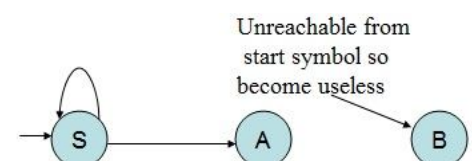


Fig. Dependency graph for Useless production

3.6 Closure Properties of CFL

The closure property of the CFL is defined itself by the CFL i.e. the language generator so it can be distinguish from the closure property of the regular set as defined earlier. The class of language of context-free language is closed under: Union, Concatenation, and Kleene closure.

A. Union

Let L_1 and L_2 be two context-free languages generated by the Context-free Grammar $G_1 = (V_1, \Sigma_1, R_1, S_1)$ and $G_2 = (V_2, \Sigma_2, R_2, S_2)$ respectively. Now, we construct new language 'L (G)' using the grammar $G = (V, \Sigma, R, S)$, such that it can accepts $L(G_1) \cup L(G_2)$.

Where,

- $V = V_1 \cup V_2 \cup \{S\}$
- $\Sigma = \text{Set of input states} = \Sigma_1 \cup \Sigma_2$
- $R = R_1 \cup R_2 \cup \{S \rightarrow S_1/S_2\}$
- $S = \text{Start state.}$

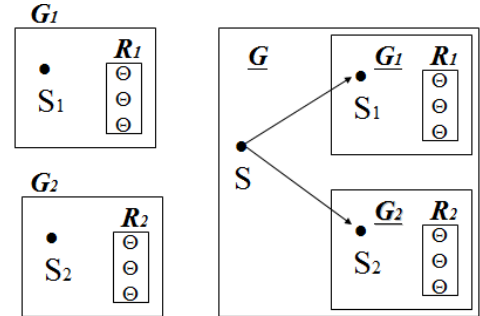


Fig. CFG is closed under union

Now, let us choose a string $w \in \{\Sigma_1 \cup \Sigma_2\}^*$ if $S_1 \xrightarrow{*} w$ and $S_2 \xrightarrow{*} w$ and in our grammar $S \rightarrow S_1/S_2$, so S will lead the string w.

Hence, G is a context-free grammar that can generate L (G) such that $L(G) = L(G_1) \cup L(G_2)$.

B. Concatenation

Let L_1 and L_2 be two context-free languages generated by the Context-free Grammar $G_1 = (V_1, \Sigma_1, R_1, S_1)$ and $G_2 = (V_2, \Sigma_2, R_2, S_2)$ respectively. Now, we construct new language 'L (G)' using the grammar $G = (V, \Sigma, R, S)$, such that it can accepts $L(G_1) \cup L(G_2)$.

Where,

- $V = V_1 \cup V_2 \cup \{S\}$
- $\Sigma = \text{Set of input states} = \Sigma_1 \cup \Sigma_2$
- $R = R_1 \cup R_2 \cup \{S \rightarrow S_1.S_2\}$
- $S = \text{Start state.}$

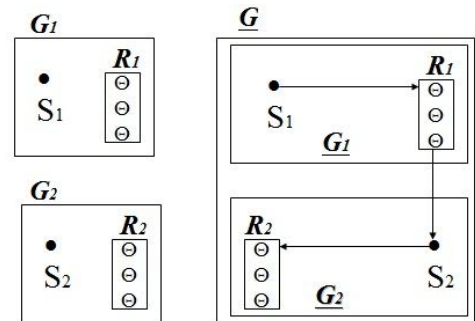


Fig. CFL is closed under Concatenation

Now, let us choose string $w_1 \in \Sigma_1$ and $w_2 \in \Sigma_2$. We know that $S_1 \xrightarrow{*} w_1$ and $S_2 \xrightarrow{*} w_2$, but in the above grammar G, $S \rightarrow S_1.S_2$, so S will lead the concatenation of the string w_1 & w_2 i.e. $w_1 w_2$ and the language will be $L_1 L_2$. Since L_1 & L_2 are CFL so $L_1 L_2$ is also CFL.

C. Kleene closure

Let L_1 be a context-free languages generated by the Context-free Grammar $G_1 = (V_1, \Sigma_1, R_1, S_1)$. Now, we construct new language 'L (G)' using the grammar $G = (V, \Sigma, R, S)$, such that it can accepts Kleene Star of the language L (or L^*).

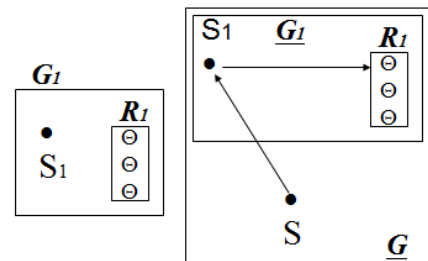


Fig. CFL is closed under Kleene Closure

Where,

- $V = V_1 \cup \{S\}$
- $\Sigma = \text{Set of input states} = \Sigma_1 \cup \{\wedge\}$
- $R = R_1 \cup \{S \rightarrow S_1, S \rightarrow \wedge, S \rightarrow S_1 S_1\}$
- $S = \text{Start state.}$

Here, R follows all the properties of CFG as R_1 is a production of given CFG and $S \rightarrow S_1$, $S \rightarrow \wedge$, & $S \rightarrow S_1 S_1$ also fulfill the requirement so we say that G is a CFG that can generate the context-free language L^* .

Decision Algorithm for Context-Free Language

- Algorithm for deciding whether a CFL is empty (*emptiness*).
- Algorithm for deciding whether a CFL is finite (*finiteness*).
- Algorithm for deciding whether any string 'w' can be generated by the same CFG (Membership).

3.7 Pumping Lemma for Context-Free Languages

The pumping lemma for the context-free language (called *Bar-Hillel lemma* or just "*the pumping lemma*") gives a method of generating an infinite number of strings from a given sufficiently long string in a context-free language L . It is used to prove that certain languages are not context-free.

While the pumping lemma is often a useful tool to prove that a given language is not context-free, it does not give a complete characterization of the context-free languages. If a language does not satisfy the condition given by the pumping lemma, we have established that it is not context-free. On the other hand, there are languages that are not context-free, but still satisfy the condition given by the pumping lemma. There are more powerful proof techniques available, such as **Ogden's lemma**, but also these techniques do not give a complete characterization of the context-free languages.

Statement

Let L be a context-free language and n be the length of the string or *pumping length* such that:

- Every $z \in L$ with $|z| = n$ can be written as $uvwxy$ for some u, v, w, x & y .** (i.e. any string z can be decompose into five sub-strings u, v, w, x & y)
- $|vx| \geq 1$** (i.e. may one null at a time but not both or must have at least one string because we have to pump at least one sub-string to generate the new infinite number of string)
- $|vwx| \leq n$** (i.e. if u & y are $\wedge$, then $vwx = n$)
- $uv^kwx^ky \in L$ for all $k \geq 0$** (i.e. generate infinite number of string by setting any value of k)

Proof

To prove the theorem, we consider a CFG whose productions are given by: $S \rightarrow AB$, $A \rightarrow aB/a$, $B \rightarrow bA/b$.

Now, let any string $z = ababb$ such that $z \in L$. Thus we can decompose z as $u = a$, $v = ba$, $w = \wedge$ and $y = b$.

Since, the string z and z_1 are the yield of the tree T . Thus, we can write: $z = uz_1vy$

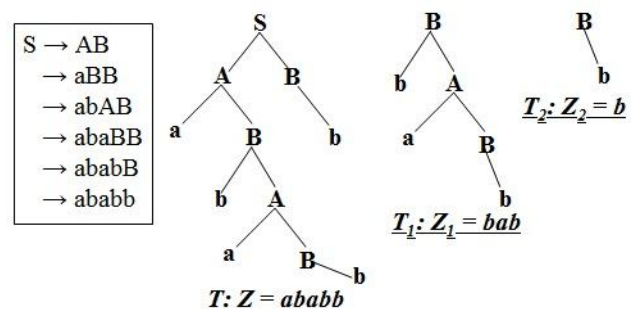


Fig. Derivation & tree T and it's sub-trees T_1 & T_2

Again, the string z_1 & z_2 are the yield of the tree T_1 . Thus, we can write: $z_1 = vwx$. Also, $|vwx| > |w|$ so $|vx| \geq 1$

Hence, we have

$z = uvwxy$ with $|vwx| \leq n$ & $|vx| \geq 1$

As T is an S-tree and T_1 & T_2 are the B-tree, we get $S \xrightarrow{*} uBy, B \xrightarrow{*} vBx, \& B \xrightarrow{*} w$

Now,

$S \rightarrow uBy$

$\rightarrow uvBxy$ (since, $B \rightarrow uBx$)

$\rightarrow uvwxy$ (since, $B \rightarrow w$)

Thus, $S \rightarrow uv^1wx^1y$, where $k = 1$

Similarly,

$S \rightarrow uBy$

$\rightarrow uvBxy$ (since, $B \rightarrow uBx$)

$\rightarrow uvuBxxy$ (since, $B \rightarrow uBx$)

$\rightarrow uvvwxy$ (since, $B \rightarrow w$)

Thus, $S \rightarrow uv^2wx^2y$, where $k = 2$ and so on.

Hence, we can conclude that $S \rightarrow uBy$ gives $S \xrightarrow{*} uv^kwx^ky \in L$ This proves the theorem.

Example: Prove that the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context-free language.

Solution

Let us consider 'L' is a context-free language. Also, let $z = a^p b^p c^p, z \in L$.

Now, according to the pumping lemma of the CFL, the string 'z' can be decomposes into u, v, w, x, & y as follow:

$$u = a^r$$

$$v = a^s \ (s > 0)$$

$$w = a^{p-(r+s)}$$

$$x = b^t \ (t > 0)$$

$$y = b^{(p-t)} c^p$$

Now, using the pumping lemma, $z = uv^kwx^ky; k \geq 0$, we have

$$z = uv^2wx^2y = a^r (a^s)^2 a^{p-(r+s)} (b^t)^2 b^{(p-t)} c^p = a^r a^{2s} a^{p-(r+s)} b^{2t} b^{(p-t)} c^p = a^{(p+s)} b^{(p+t)} c^p \neq a^p b^p c^p$$

Here, our assumption $z \in L$, contradict with our result.

Hence, we can conclude that the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context-free language.

Example: Prove that the language $L = \{a^n \mid n \geq 0\}$ is not context-free language.

Solution

Let us consider 'L' is a context-free language. Also, let $z = a^p, z \in L$.

Now, according to the pumping lemma of the CFL, the string 'z' can be decomposes into u, v, w, x, & y as follow:

$$u = a^\alpha, v = a^\beta \ (\beta > 0), w = a^{q-(\alpha+\beta)}, x = a^\gamma \ (\gamma > 0), \& y = a^{p-(q+\gamma)}$$

Now, using the pumping lemma, $z = uv^kwx^ky; k \geq 0$, we have

$$z = uv^2wx^2y = a^\alpha (a^\beta)^2 a^{q-(\alpha+\beta)} (a^\gamma)^2 a^{p-(q+\gamma)} = a^\alpha a^{2\beta} a^{q-(\alpha+\beta)} a^{2\gamma} a^{p-(q+\gamma)} = a^{p+\alpha+\beta} \neq a^p$$

Here, our assumption $z \in L$, contradict with our result.

Hence, we can conclude that the language $L = \{a^n \mid n \geq 0\}$ is not context-free language.

3.8 Pushdown Automata

Finite automata provide only finite amount of memory, thus preventing us from recognizing language that require remembering an infinite amount of information during the processing of a string. For example a finite automaton cannot accept the string of the form $L = a^n b^n$ ($n \geq 1$) as it has to remember the number of a's in a string and so it will require infinite number of states. This difficulty can be avoided by adding memory capability to finite automata. The addition of infinite stack to the finite automata leads to what we call the *push down automata* (pda).

Thus, the push down automata is essentially a finite automaton with control of both input tape and a stack to store what it has read. Hence, the pushdown automata consist of the following three things:

- An input tape
- A finite control
- A stack

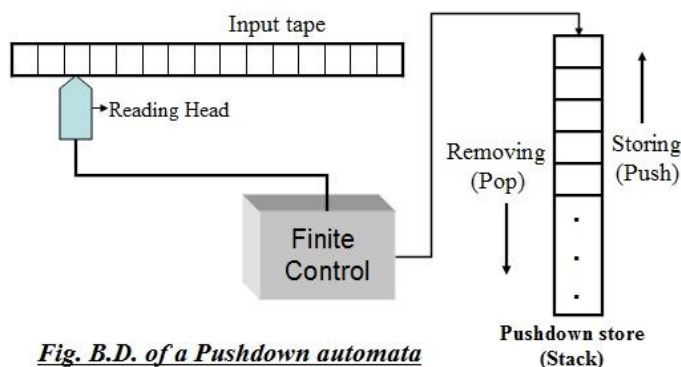


Fig. B.D. of a Pushdown automata

Here is an input tape from which the finite control reads the input and same time it reads the symbol from the stack. Now it depends upon the finite control that what is the next state and what will happen with the stack. In one transition, the pushdown automata do the following:

- Consume the input symbol that it uses in the transition. If the input is $\wedge$, then there is no consumption of input symbol.
- Goes to the new state, which may or may not be the same as the previous state.
- Replace the symbol at the top of the stack by any string. It could be the same symbol that appeared at the top of the stack.

Mathematically, the pushdown automata, is six tuple, can be defined as $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$. Where

- Q is the non-empty finite set of states.
- Σ is the non-empty finite set of input symbol.
- Γ is a finite non-empty set of pushdown or stack symbol
- q_0 is the start state, $q_0 \in Q$.
- F is a set of final states, $F \subseteq Q$.
- δ is a transition function which maps $(Q \times \Sigma^* \times \Gamma^*) \rightarrow (Q \times \Gamma^*)$. Formally δ takes an argument, a triple as $\delta(q, a, x)$ where,
 - q is a state in Q .
 - a is an input symbol.
 - x is a stack symbol that is a member of Γ .

The output of δ is a finite set of pair (p, r) , where 'p' is the new state and r is the string of the stack symbol that replaces x at the top of the stack.

Move of PDA

The pushdown automata consist of following moves:

- * $\delta(q, a, x) \rightarrow (p, y)$ means that whenever PDA is in state q with x, the top of stack, may read 'a' from the input tape, replace 'x' by 'y' on the top of stack and enter state p.
- * $\delta(q, a, \wedge) \rightarrow (p, y)$, indicates the pushes 'y' on the top of the stack.
- * $\delta(q, a, y) \rightarrow (p, \wedge)$, indicates the pops a symbol 'y' from the top of the stack.

Example: Design a PDA which accepts a language $L = \{w = a^n b^n : n \geq 1\}$

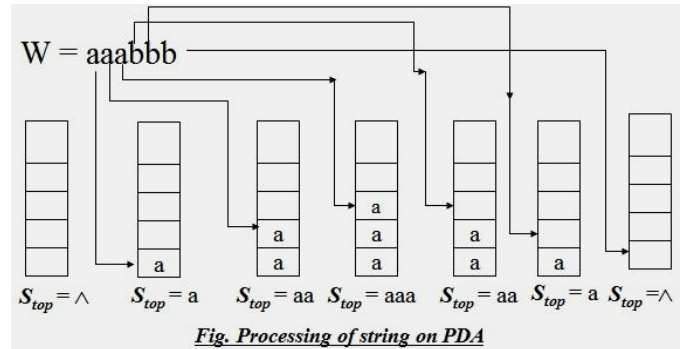
Solution

To solve the problem, we have to first analyze the given language so that we can generate the PDA for it. As we can see, the string must have equal number of 'a' and 'b' and the order of placement is number of 'a' followed by number of 'b'. Thus, to design such PDA, we have to read the number of 'a' and then the same number of 'b' and finally, the input string and the stack must be empty for the accepting condition. To do so, we have to push first the whole string of 'a', one by one, into the empty stack and when the string of 'b' start to read, pop one by one the string of 'a' on stack in each 'b' read. Hence, when the input string is consumed, there is nothing on the stack also and that we termed as the accepting condition of the PDA.

Now, let the required PDA for the given string be: $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$, Where

- Q = finite set of states = $\{q_0, q_1, q_f\}$
- Σ = finite set of input symbol = $\{a, b\}$
- Γ = finite set of stack symbol = $\{a\}$
- q_0 = start state
- F = set of final states = $\{q_f\}$
- δ = transition function which can be defined as

- i. $\delta(q_0, a, \wedge) \rightarrow (q_1, a)$
- ii. $\delta(q_1, a, a) \rightarrow (q_1, aa)$
- iii. $\delta(q_1, b, a) \rightarrow (q_1, \wedge)$
- iv. $\delta(q_1, \wedge, \wedge) \rightarrow (q_f, \wedge)$



For example, let $w = aaabbb$ is any string then we can process it using PDA defined above in tabular form:

Present States	Unread Input	Present Stack symbols	Next States	Next Stack symbols	Transition Used
q_0	$\rightarrow aaabbb$	$\wedge$	q_1	a	I
q_1	$\rightarrow aabbb$	a	q_1	aa	II
q_1	$\rightarrow abbb$	aa	q_1	aaa	II
q_1	$\rightarrow bbb$	aaa	q_1	aa	III
q_1	$\rightarrow bb$	aa	q_1	a	III
q_1	$\rightarrow b$	a	q_1	$\wedge$	III
q_1	$\rightarrow \wedge$	$\wedge$	q_f		IV

Example: Design a PDA which accepts a language $L = \{w \in \{a, b\}^* \text{ where } w \text{ has equal number of } a \text{ \& } b\}$.

Solution

As we can see, the string must have equal number of 'a' and 'b' with any order of placement. Hence, let us consider $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be the required PDA for the given string where

- Q = finite set of states = $\{q_0, q_f\}$
 - Σ = finite set of input symbol = $\{a, b\}$
 - Γ = finite set of stack symbol = $\{a, b\}$
 - q_0 = start state
 - F = set of final states = $\{q_f\}$
 - δ = transition function which can be defined as
- i. $\delta(q_0, a, \wedge) \rightarrow (q_0, a)$
 - ii. $\delta(q_0, b, \wedge) \rightarrow (q_0, b)$

- iii. $\delta(q_0, a, a) \rightarrow (q_0, aa)$
- iv. $\delta(q_0, b, b) \rightarrow (q_0, bb)$
- v. $\delta(q_0, a, b) \rightarrow (q_0, \wedge)$
- vi. $\delta(q_0, b, a) \rightarrow (q_0, \wedge)$
- vii. $\delta(q_0, \wedge, \wedge) \rightarrow (q_f, \wedge)$

For example, let $w = abbbaaba$ is any string then we can process it using PDA defined above in tabular form:

<i>Present States</i>	<i>Unread Input</i>	<i>Stack top</i>	<i>Next States</i>	<i>New top</i>	<i>Stack-Used</i>
q_0	$\rightarrow abbbaaba$	$\wedge$	q_0	a	I
q_0	$\rightarrow bbbaaba$	a	q_0	$\wedge$	VI
q_0	$\rightarrow bbaaba$	$\wedge$	q_0	b	II
q_0	$\rightarrow baaba$	b	q_0	bb	IV
q_0	$\rightarrow aaba$	bb	q_0	b	V
q_0	$\rightarrow aba$	b	q_0	$\wedge$	V
q_0	$\rightarrow ba$	$\wedge$	q_0	b	II
q_0	$\rightarrow a$	b	q_0	$\wedge$	V
q_0	$\rightarrow \wedge$	$\wedge$	q_f	$\wedge$	VII

This can also be done in the following manner:

- $\delta(q_0, \text{a}bbbaaba, \wedge) \rightarrow (q_0, a)$ by rule I.
- $\delta(q_0, \text{b}bbaaba, a) \rightarrow (q_0, \wedge)$ by rule VI.
- $\delta(q_0, \text{b}baaba, \wedge) \rightarrow (q_0, b)$ by rule II.
- $\delta(q_0, \text{b}aaba, b) \rightarrow (q_0, bb)$ by rule IV.
- $\delta(q_0, \text{a}aba, bb) \rightarrow (q_0, b)$ by rule V.
- $\delta(q_0, \text{a}ba, b) \rightarrow (q_0, \wedge)$ by rule V.
- $\delta(q_0, \text{b}a, \wedge) \rightarrow (q_0, b)$ by rule II.
- $\delta(q_0, \text{a}, b) \rightarrow (q_0, \wedge)$ by rule V.
- $\delta(q_1, \wedge, \wedge) \rightarrow (q_f, \wedge)$, by rule VII, which is the accepting state.

Example: Design a PDA which accepts a language $L = (wcw^T, w \in \{a, b\}^*)$ i.e. w belong to any strings of a & b).

Solution

As we can see, the string of any number of 'a' or 'b' must be complement on both side of 'c'. Hence, let us consider that $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be the required PDA for the given string where

- Q = finite set of states = $\{q_0, q_1, q_f\}$
- Σ = finite set of input symbol = $\{a, c, b\}$
- Γ = finite set of stack symbol = $\{a, b\}$
- q_0 = start state
- F = set of final states = $\{q_f\}$
- δ = transition function which can be defined as
 - i. $\delta(q_0, a, \wedge) \rightarrow (q_0, a)$
 - ii. $\delta(q_0, a, a) \rightarrow (q_0, aa)$
 - iii. $\delta(q_0, a, b) \rightarrow (q_0, ab)$
 - iv. $\delta(q_0, b, b) \rightarrow (q_0, bb)$
 - v. $\delta(q_0, b, a) \rightarrow (q_0, ba)$
 - vi. $\delta(q_0, c, a) \rightarrow (q_1, a)$ i.e. only state change.
 - vii. $\delta(q_0, c, b) \rightarrow (q_1, b)$ i.e. only state change.

- viii. $\delta(q_1, a, a) \rightarrow (q_1, \wedge)$
- ix. $\delta(q_1, b, a) \rightarrow (q_1, \wedge)$
- x. $\delta(q_1, a, b) \rightarrow (q_1, \wedge)$
- xi. $\delta(q_0, b, \wedge) \rightarrow (q_0, b)$
- xii. $\delta(q_1, b, b) \rightarrow (q_1, \wedge)$
- xiii. $\delta(q_1, \wedge, \wedge) \rightarrow (q_f, \wedge)$, which is the accepting state.

For example, let $w = aabcbbaa$ is any string then we can process it using PDA defined above in following tabular form:

Present States	Unread Input	Stack top	Next States	New Stack-top	Transition Used
q_0	$\rightarrow aabcbbaa$	$\wedge$	q_0	a	I
q_0	$\rightarrow abcbbaa$	a	q_0	aa	II
q_0	$\rightarrow bcbbaa$	aa	q_0	$bbaa$	VIII
q_0	$\rightarrow cbbaa$	$bbaa$	q_1	$bbaa$	IX
q_1	$\rightarrow bbaa$	$bbaa$	q_1	aa	X
q_1	$\rightarrow aa$	aa	q_1	a	V
q_1	$\rightarrow a$	a	q_1	$\wedge$	V
q_1	$\rightarrow \wedge$	$\wedge$	q_f	$\wedge$	XI

Example: Design a PDA which accepts a language $L = \{a^n b^{2n}, n > 0\}$.

Solution

As we can see, the string must have just half number of 'a' than that of 'b' with order of placement is 'a' followed by 'b'. Hence, let us consider $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be the required PDA for the given string where

- Q = finite set of states = $\{q_0, q_1, q_f\}$
- Σ = finite set of input symbol = $\{a, b\}$
- Γ = finite set of stack symbol = $\{a\}$
- q_0 = start state
- F = set of final states = $\{q_f\}$
- δ = transition function which can be defined as
 - i. $\delta(q_0, a, \wedge) \rightarrow (q_1, aa)$
 - ii. $\delta(q_1, a, a) \rightarrow (q_1, aaa)$
 - iii. $\delta(q_1, b, a) \rightarrow (q_1, \wedge)$
 - iv. $\delta(q_1, \wedge, \wedge) \rightarrow (q_f, \wedge)$

For example, let $w = aabbbb$ is any string then we can process it using PDA defined above in following tabular form:

Present States	Unread Input	Present Stack symbols	Next States	Next Stack symbols	Transition Used
q_0	$\rightarrow aabbbb$	$\wedge$	q_1	aa	I
q_1	$\rightarrow abbbb$	aa	q_1	$aaaa$	II
q_1	$\rightarrow bbbb$	$aaaa$	q_1	aaa	II
q_1	$\rightarrow bbb$	aaa	q_1	aa	III
q_1	$\rightarrow bb$	aa	q_1	a	III
q_1	$\rightarrow b$	a	q_1	$\wedge$	III
q_1	$\rightarrow \wedge$	$\wedge$	q_f		IV

Exercise: -Design a PDA which accepts a language $L = \{a^n b^{n+1}, n > 0\}$.

Hind: - We can construct the string of the form $L = a^n bb^n$

3.9 Context-Free Grammar (CFG) to Pushdown Automata (PDA) Conversation

The CFG generates the language accepted by the PDA so we can construct the PDA equivalent to the given Context-Free Grammar.

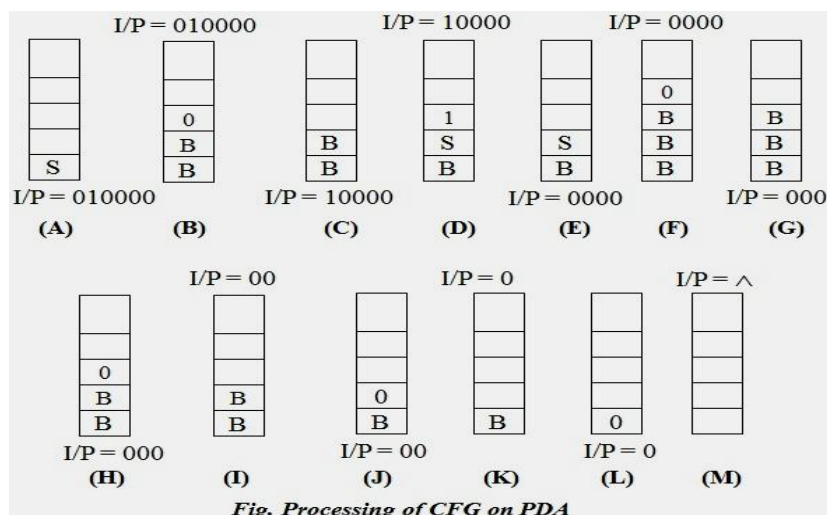
Example: Construct a PDA equivalent to the following CFG: $S \rightarrow 0BB, B \rightarrow 0S/1S/0$. Also, test whether 010000 is accepted by PDA or not.

Solution

We can define the PDA as: $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ where

- Q = finite set of states = $\{q_0, q_f\}$
 - Σ = finite set of input symbol = $\{0, 1\}$
 - Γ = finite set of stack symbol = $\{S, B, 0, 1\}$
 - q_0 = start state
 - F = set of final states = $\{q_f\}$
 - δ = transition function which can be defined by the following rules:
 - i. $\delta(q_0, \wedge, \wedge) \rightarrow (q_f, S)$
 - ii. $\delta(q_f, \wedge, S) \rightarrow (q_f, 0BB)$
 - iii. $\delta(q_f, \wedge, B) \rightarrow \{(q_f, 0S), (q_f, 1S), (q_f, 0)\}$
 - iv. $\delta(q_f, 0, 0) \rightarrow (q_f, \wedge)$
 - v. $\delta(q_f, 1, 1) \rightarrow (q_f, \wedge)$
- For the given string $w = 010000$, we can process it using the defined transition rules.

▪ On the basis of stack processing



▪ **Tabular form**

<i>Present States</i>	<i>Unread Input</i>	<i>Stack top</i>	<i>Next States</i>	<i>New Stack-top</i>	<i>Transition Used</i>
q_0	$\rightarrow \wedge 010000$	$\wedge$	q_f	S	I
q_f	$\rightarrow \wedge 010000$	S	q_f	0BB	II
q_f	$\rightarrow 010000$	0BB	q_f	BB	IV
q_f	$\rightarrow \wedge 10000$	BB	q_f	1SB	III
q_f	$\rightarrow 10000$	1SB	q_f	SB	V
q_f	$\rightarrow \wedge 0000$	SB	q_f	0BBB	II
q_f	$\rightarrow 0000$	0BBB	q_f	BBB	IV
q_f	$\rightarrow \wedge 000$	BBB	q_f	0BB	III
q_f	$\rightarrow 000$	0BB	q_f	BB	IV
q_f	$\rightarrow \wedge 00$	BB	q_f	0B	III
q_f	$\rightarrow 00$	0B	q_f	B	IV
q_f	$\rightarrow \wedge 0$	B	q_f	0	III
q_f	$\rightarrow 0$	0	q_f	$\wedge$	IV
	$\rightarrow \wedge$	$\wedge$	q_f		

▪ This can also be done in the following manner: -

- $\delta(q_0, 010000, \wedge) \rightarrow \delta(q_f, 010000, 0BB) \rightarrow \delta(q_f, 10000, BB)$
 $\rightarrow \delta(q_f, 10000, 1SB) \rightarrow \delta(q_f, 0000, SB)$
 $\rightarrow \delta(q_f, 0000, 0BBB) \rightarrow \delta(q_f, 000, BBB)$
 $\rightarrow \delta(q_f, 000, 0BB) \rightarrow \delta(q_f, 00, BB)$
 $\rightarrow \delta(q_f, 00, 0B) \rightarrow \delta(q_f, 0, B)$
 $\rightarrow \delta(q_f, 0, 0)$
 $\rightarrow \delta(q_f, \wedge, \wedge)$, which is the accepting condition.